

**Ben-Gurion University of the Negev**  
**Digital Design and Logic Synthesis (361-1-3611)**

# **Final Project Report**

**RTL Design Verification & synthesis of AES Modules**

**Submitted by:** Adi Shlomo 211619226, Shahar Halevy 209058114

**Date:** January 2026

# Table of contents

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 1. Introduction .....                                                     | 3  |
| 1.1 Design & Architecture .....                                           | 3  |
| 1.2 Verification Environment.....                                         | 3  |
| 1.3 Implementation & Results.....                                         | 3  |
| 2. Logical Design & Architecture .....                                    | 4  |
| 2.1 High-Level Architecture .....                                         | 4  |
| 2.2 FSM State Diagram .....                                               | 5  |
| 2.3 Block Diagram .....                                                   | 7  |
| 2.3.1 Top-Level Interface .....                                           | 7  |
| 2.3.2 Detailed Schematic .....                                            | 8  |
| 3. Verification Environment Architecture .....                            | 9  |
| 3.1 Environment Block Diagram .....                                       | 9  |
| 3.2 Verification Environment Overview .....                               | 10 |
| 4. Verification Results .....                                             | 11 |
| 4.1 Simulation Log: Stimuli, Reference Model & Scoreboard Validation .... | 11 |
| 4.1.1 Stimulus Strategy & Regression Summary.....                         | 11 |
| 4.1.2 Golden Model & Scoreboard Alignment .....                           | 11 |
| 4.1.3 Result Analysis .....                                               | 11 |
| 4.2 Waveform Analysis .....                                               | 13 |
| 4.3 Verification Metrics: Coverage & Assertions.....                      | 14 |
| 4.3.1 Detailed Functional Coverage .....                                  | 15 |
| 4.3.2 Protocol Validation (Assertions) .....                              | 17 |
| 4.3.3 Code Coverage Analysis .....                                        | 18 |
| 5. Synthesis .....                                                        | 20 |
| 5.1 Synthesis Planning .....                                              | 20 |
| 5.2 Timing Performance .....                                              | 22 |
| 5.3 Power & Area Efficiency (Clock Gating Analysis) .....                 | 24 |
| 5.4 Gate-Level Simulation (GLS).....                                      | 27 |
| 5.5 Runtime and Resource Utilization .....                                | 29 |
| 6. Conclusion .....                                                       | 31 |

# 1. Introduction

This document serves as the Final Report for the AES Hardware Accelerator project, covering the full VLSI frontend flow from RTL design and verification to synthesis, timing optimization, and GLS. The course-scoped implementation focuses on the AES-128 SubBytes (S-Box) and MixColumns modules, integrated under an FSM-controlled Start/Busy/Done handshake and targeted to a 90nm technology node.

## 1.1 Design & Architecture

The system implements core AES-128 datapath transformations (SubBytes and MixColumns) using a modular, pipelined architecture designed for high throughput. Key architectural features include:

- **Optimized Datapath:** Dedicated hardware modules for SubBytes and MixColumns transformations.
- **Robust Control:** A finite state machine (FSM) managing a strict handshake protocol (Start/Busy/Done) to ensure data integrity.
- **Design for Synthesis:** The RTL was structured specifically to support logic optimization techniques such as register retiming.

## 1.2 Verification Environment

To reduce verification risk and increase confidence in correctness, a rigorous Verification Environment was developed using SystemVerilog. This environment employs industry-standard methodologies, including:

- **Constrained Random Verification (CRV)** for high scenario coverage.
- **Self-Checking Scoreboards** comparing results against a Golden Reference Model.
- **Functional Coverage & Assertions** ensuring 100% protocol compliance and state-space coverage.

## 1.3 Implementation & Results

The culmination of the project is the physical mapping of the verified design using Synopsys Design Compiler. Through advanced synthesis strategies, the final Gate-Level Netlist achieved exceptional performance metrics:

- **Operating Frequency: 416 MHz** (2.4ns clock period) with full timing closure (Zero Slack).
- **Power Efficiency: ~79.4% reduction** in dynamic power consumption via Integrated Clock Gating (ICG).
- **Functional Validity:** Confirmed via Post-Synthesis Gate-Level Simulation (GLS) with SDF back-annotation.

## 2. Logical Design & Architecture

In this phase, the hardware architecture was refined to support Advanced Verification methodologies. The design now incorporates a robust Handshake Protocol and input protection mechanisms to ensure deterministic behavior in a simulation environment.

### 2.1 High-Level Architecture

The Data Path was upgraded to include an Input Register (u\_input\_reg) at the top level.

- **Input Latching:** This register samples the data\_in bus only when the transaction begins (triggered by the FSM). This "snapshot" ensures that the internal processing pipeline (SubBytes → MixColumns) works on stable data, even if the external input bus changes or becomes noisy during the calculation.
- **Pipeline Flow:** The data flows linearly from the Input Register to the SubBytes module and then to the MixColumns module.
- **Visibility:** For verification purposes, the intermediate output of the SubBytes stage (sb\_out) is exposed as a top-level port, allowing the Testbench to verify stage-by-stage correctness.

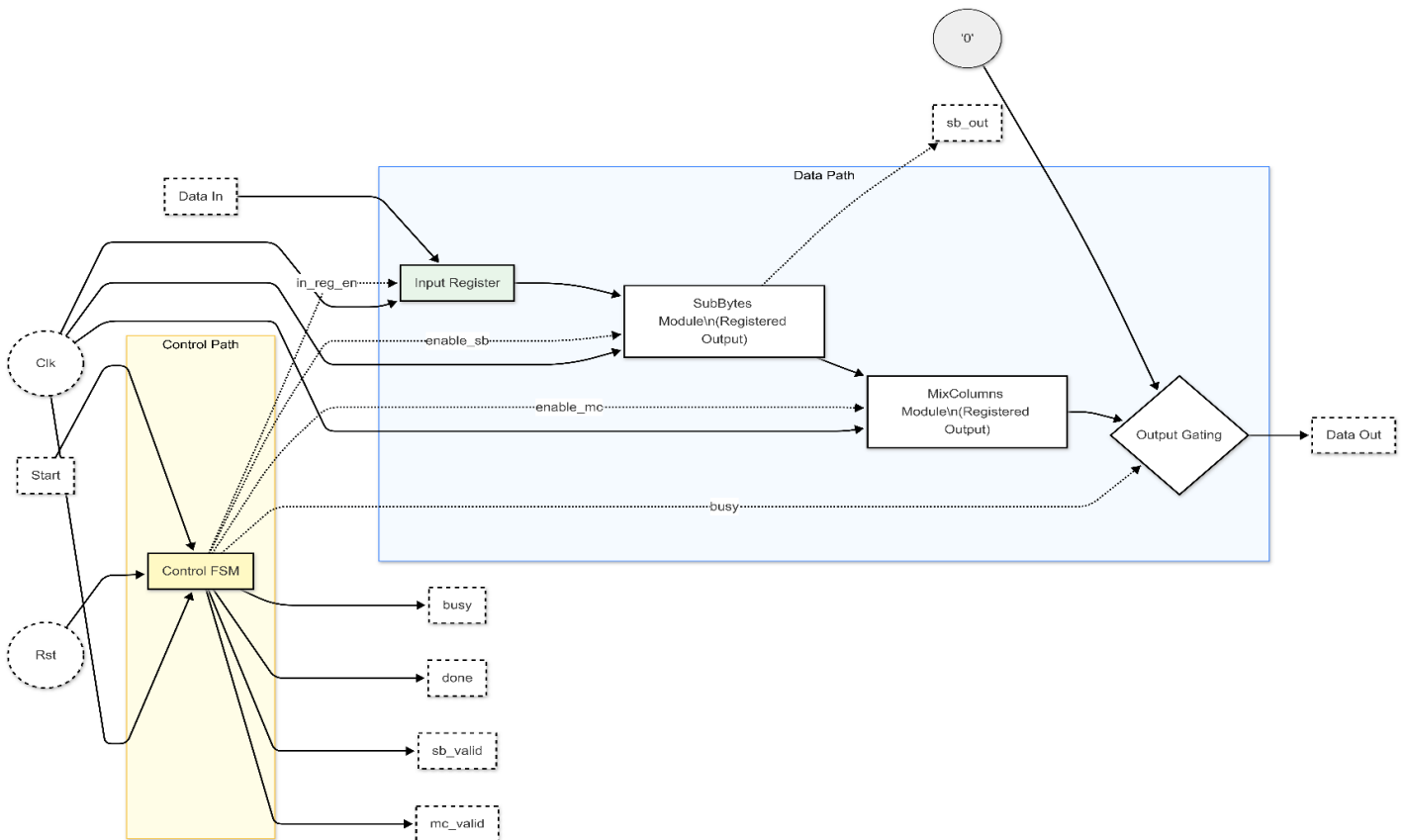


Figure 1: High-Level Logical Block Diagram

## 2.2 FSM State Diagram

The system control utilizes a synchronous Moore FSM with 4 defined states. A key enhancement in Part 2 is the Output Signal Synchronization:

- The sb\_valid and done flags are generated by registering the enable signals. This creates a 1-cycle delay, ensuring that the valid flags are asserted exactly when the data is available at the output of the synchronous modules.

### State Descriptions:

1. **RESET (00):** System initialization.
2. **IDLE (01):** The system waits for the start signal. When received, it enables the Input Register to capture data.
3. **SB (10):** SubBytes calculation. en\_sb is active.
4. **MC (11):** MixColumns calculation. en\_mc is active. At this cycle, the SubBytes result is ready, so sb\_valid is asserted (delayed from previous state).

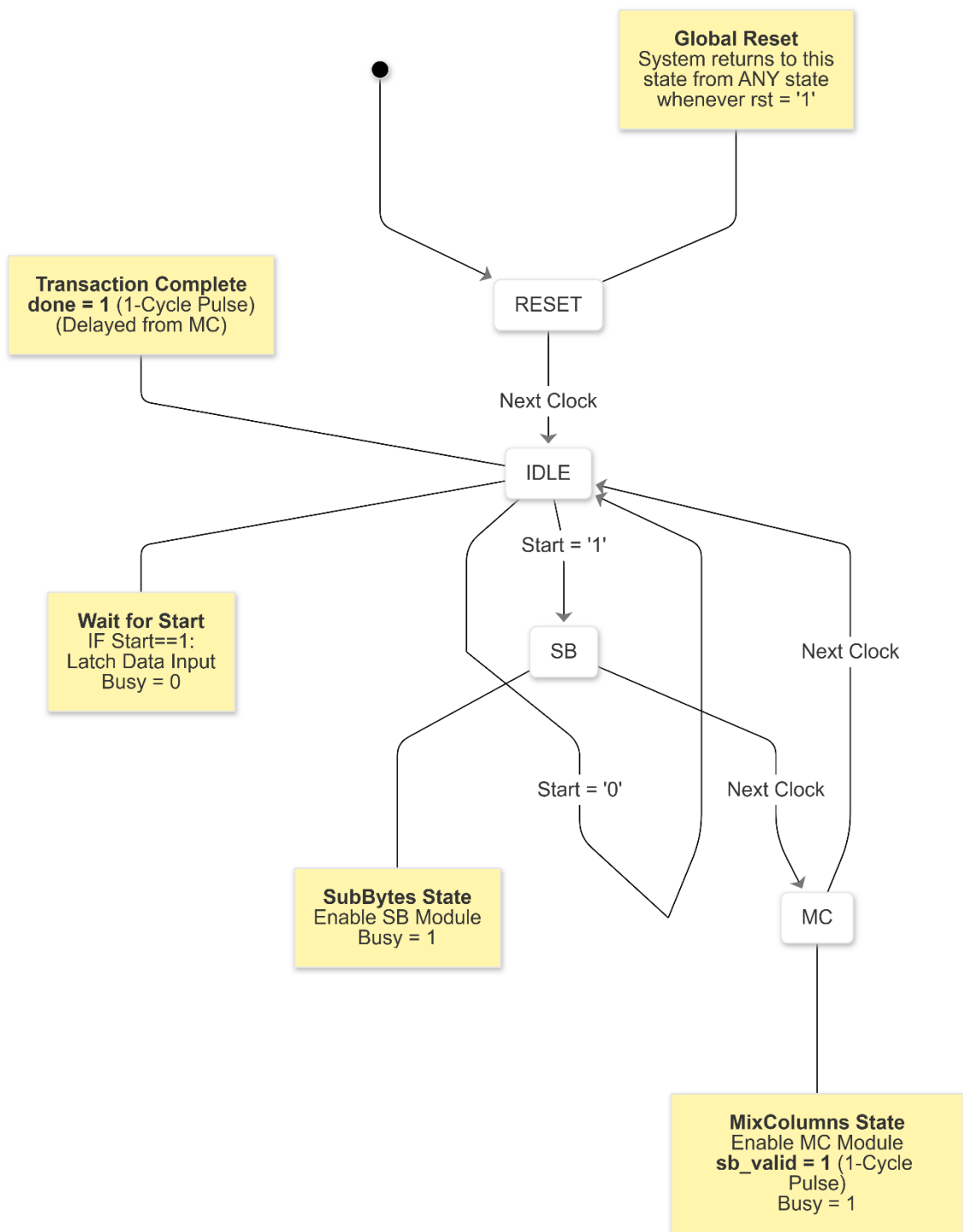


Figure 2: FSM State Transition Diagram

## 2.3 Block Diagram

### 2.3.1 Top-Level Interface

Figure 3 presents the interface of the top-level module. It highlights the 128-bit input/output data buses (data\_in, data\_out) and the main control signals (clk, rst, start). In addition, the top level exposes status/observability signals used by the verification environment, including busy, done, sb\_valid, fsm\_state, and sb\_out.

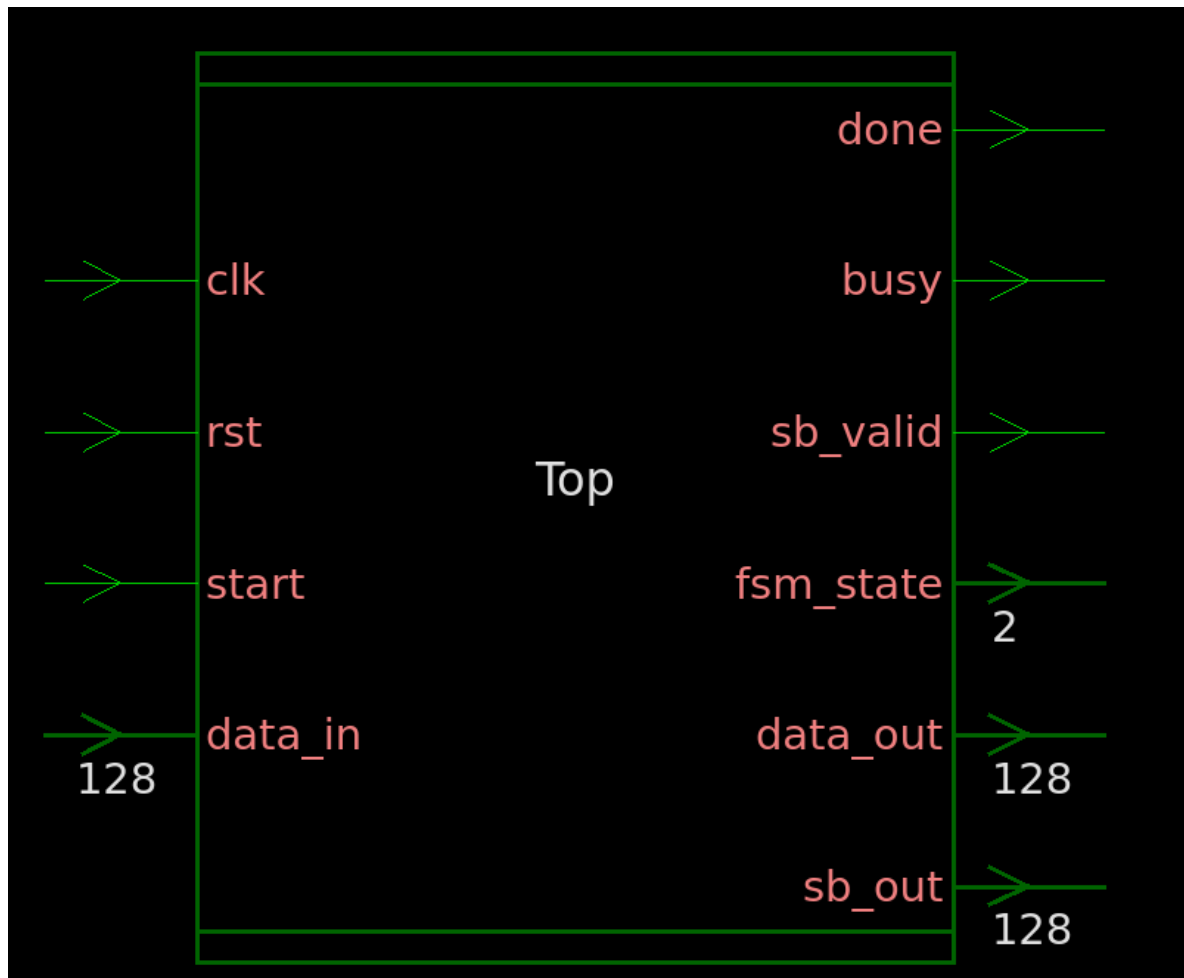


Figure 3: Top-Level Module Interface

### 2.3.2 Detailed Schematic

Figure 4 shows the internal hierarchical structure of the synthesized design and the major connectivity between the controller (u\_control\_fsm) and the datapath modules. The schematic illustrates how the FSM sequences the operation and how intermediate 128-bit state values are propagated through the main AES processing stages (e.g., SubBytes/MixColumns) via internal registers.

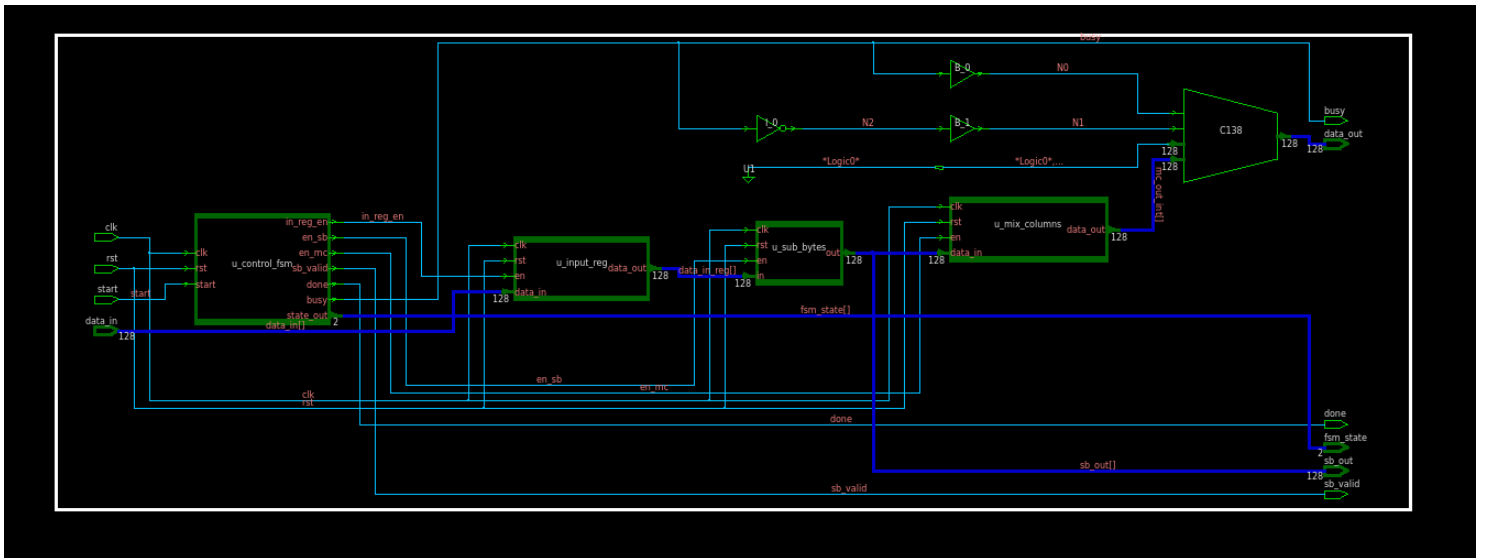


Figure 4: Top-Level Synthesized Schematic (hierarchical view and controller-to-datapath connectivity)



### 3. Verification Environment Architecture

Unlike traditional directed testbenches, this project utilizes a Layered Verification Environment based on SystemVerilog classes. This approach promotes reusability, scalability, and better abstraction of the verification logic.

#### 3.1 Environment Block Diagram

The following diagram illustrates the architecture of the verification environment and the data flow between its components. It demonstrates the Layered Architecture approach, highlighting the separation between the high-level test logic (Generator/Scoreboard) and the low-level signal driving (BFM). The diagram also depicts the parallel analysis paths for self-checking (Scoreboard) and functional coverage collection.

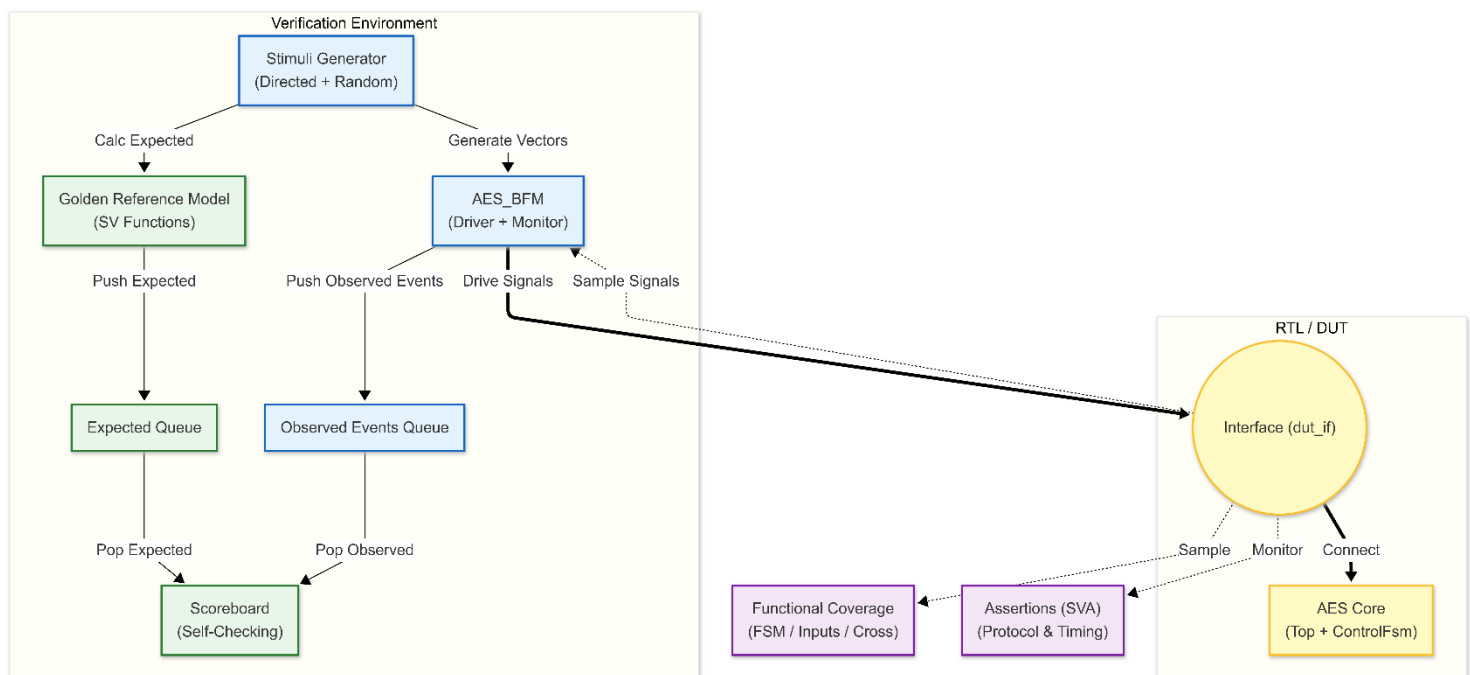


Figure 5: Verification Environment Block Diagram

## 3.2 Verification Environment Overview

The verification environment is implemented under the TB directory and follows an expected-vs-observed self-checking methodology. A stimulus generator produces transactions and drives the DUT through a BFM (driver/monitor). A behavioral golden reference model (SubBytes + MixColumns) computes expected intermediate (exp\_sb) and final (exp\_out) results. The scoreboard aligns expected transactions with observed events and performs two stage-aware checks: sb\_out on sb\_valid, and data\_out on done. Assertions enforce protocol/timing correctness, while functional coverage tracks FSM flow and key corner scenarios. Detailed regression composition and results are reported in Section 4.

## 4. Verification Results

The verification environment was executed using ModelSim/Questasim. Results demonstrate functional correctness across directed and randomized scenarios.

### 4.1 Simulation Log: Stimuli, Reference Model & Scoreboard Validation

#### 4.1.1 Stimulus Strategy & Regression Summary

The console output provides clear evidence of functional correctness, showing that the Scoreboard successfully cross-referenced each observed DUT result (on sb\_valid and done) against the Golden Reference Model in real time. The verification environment executed a regression suite of 559 transactions, utilizing a hybrid stimulus strategy to maximize coverage efficiency:

- Directed Edge-Cases (3 txns): Pre-defined vectors targeting specific boundary conditions (e.g., all-zeros, all-ones) to verify basic connectivity and reset behavior.
- Deterministic S-Box Sweep (256 txns): A loop iterating 0x00–0xFF applied simultaneously to all byte lanes. This ensures the substitution logic is exercised across the complete 0x00–0xFF input space on all byte lanes.
- Constrained-Random Verification (300 txns): Randomized payloads with randomized inter-transaction delays. This stresses the control logic by creating varying "back-to-back" and "delayed" scenarios.

#### 4.1.2 Golden Model & Scoreboard Alignment

For each input vector, the testbench computes expected results using the in-test golden reference model (SubBytes + MixColumns), generating both the expected intermediate state (exp\_sb) and final output (exp\_out). Expected transactions are queued in order, while the monitor publishes observed events on sb\_valid and done. The scoreboard performs two checks per transaction: (1) compares sb\_out on sb\_valid, and (2) compares data\_out on done, enabling precise localization of failures to the relevant pipeline stage.

#### 4.1.3 Result Analysis

The simulation concluded with 0 FAILS, demonstrating bit-accurate behavior. Notably, the Pass Count of 1118 reflects a rigorous Multi-Stage Checking strategy: For every single transaction, the Scoreboard performs two independent validations:

1. **Intermediate Check:** Verifies sb\_out against the reference model's internal state.
2. **Final Check:** Verifies data\_out upon transaction completion. This ensures that errors are caught exactly where they occur in the pipeline, rather than just observing the final result.

```
# TB COMPLETE | txns=559 pass=1118 fail=0
# ** Note: $stop : /users_home/adishl/TB/tb_top.sv(68)
# Time: 32720 ns Iteration: 0 Instance: /tb_top
# Break in Module tb_top at /users_home/adishl/TB/tb_top.sv line 68
```

Figure 6: Simulation Console Log. The summary txns=559 and pass=1118 indicates successful scoreboard checking across all test vectors, including the complete S-Box sweep.

## 4.2 Waveform Analysis

The waveform capture (Figure 7) provides a chronological view of a complete encryption transaction, validating the control logic and data integrity across the pipeline stages. The analysis is broken down into four key phases:

1. **Initialization and Idle State:** Prior to the transaction, the system is in the RESET state (indicated by `fsm_state = 0`). Upon de-assertion of `rst`, the FSM transitions to IDLE (`fsm_state = 1`). In this state, the busy signal remains low and the system waits for a trigger, while internal control signals remain inactive to maintain a known state.
2. **Transaction Initiation & Input Latching:** When the start signal is asserted, the FSM detects the request on the next rising clock edge. This triggers two simultaneous actions:
  - **Input Sampling:** The `in_reg_en` signal is asserted for a single cycle, immediately latching the value from `data_in` into the internal Input Register.
  - **State Transition:** The FSM moves to the SB (SubBytes) state (`fsm_state = 2`) and the busy flag is asserted high, indicating that the core is processing and ignoring new inputs.
3. **Pipeline Execution (SB & MC):** The processing advances through the architectural pipeline stages:
  - **SubBytes (State 2):** The `en_sb` signal is active, driving the S-Box substitution logic.
  - **MixColumns (State 3):** In the following cycle, the FSM transitions to MC (`fsm_state = 3`) and asserts `en_mc`. Crucially, the waveform shows `sb_valid` pulsing high during this state, confirming that the intermediate `sb_out` is stable and valid for the MixColumns logic.
4. **Completion and Output Validity:** Upon completion, the FSM returns to the IDLE state (`fsm_state = 1`). Synchronized with this transition, the done signal asserts for exactly one clock cycle. The waveform demonstrates that the final `data_out` is stable precisely when done is high. Following this pulse, busy is de-asserted, signifying readiness for the next transaction.

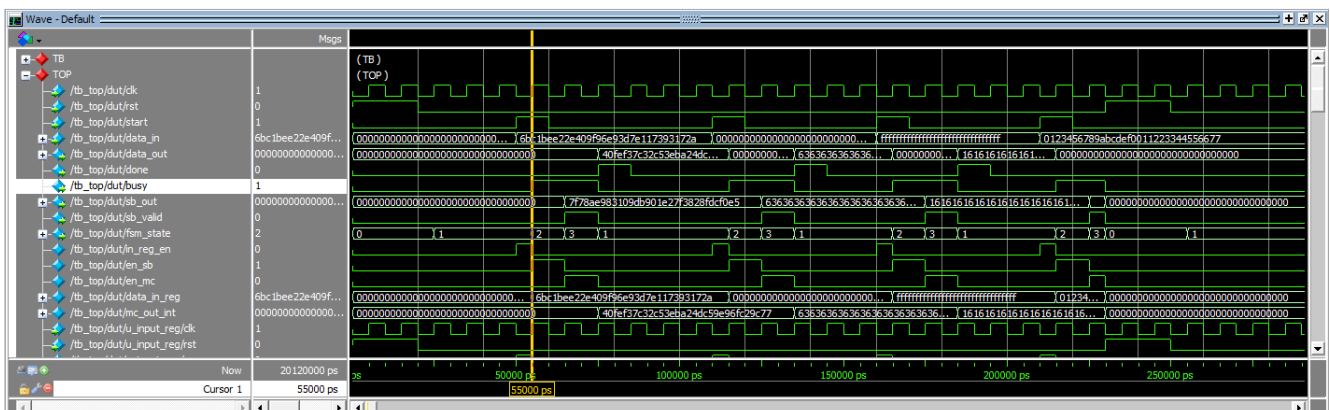
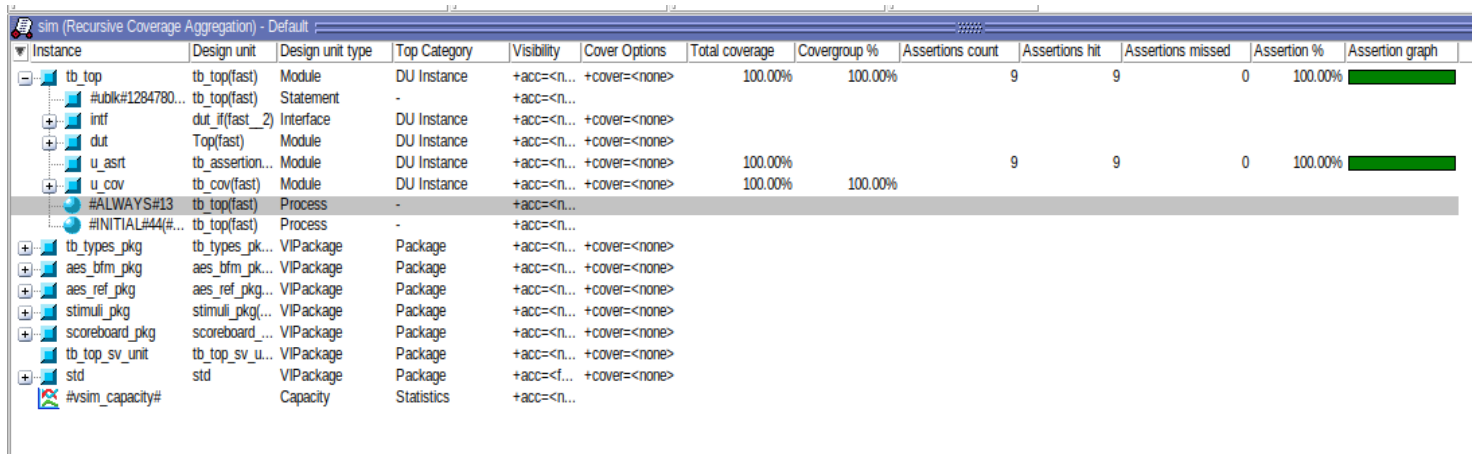


Figure 7: Waveform demonstrating start → busy → sb\_valid → done timing.

## 4.3 Verification Metrics: Coverage & Assertions

To ensure both the completeness of the test scenarios and the correctness of the protocol, the verification environment tracks two parallel metrics: Functional Coverage and SystemVerilog Assertions (SVA).

Overview of Results as shown in the simulation dashboard below, the environment achieved a 100% success rate in both categories. This confirms that all planned scenarios were executed (Coverage) and no protocol violations occurred (Assertions).



| Instance         | Design unit     | Design unit type | Top Category | Visibility               | Cover Options            | Total coverage | Covergroup % | Assertions count | Assertions hit | Assertions missed | Assertion % | Assertion graph                                                                     |
|------------------|-----------------|------------------|--------------|--------------------------|--------------------------|----------------|--------------|------------------|----------------|-------------------|-------------|-------------------------------------------------------------------------------------|
| tb_top           | tb_top(fast)    | Module           | DU Instance  | +acc=<n... +cover=<none> |                          | 100.00%        | 100.00%      | 9                | 9              | 0                 | 100.00%     |  |
| #ublk#1284780... | tb_top(fast)    | Statement        | -            | +acc=<n...               |                          |                |              |                  |                |                   |             |                                                                                     |
| +                | intf            | dut_if(fast_2)   | Interface    | DU Instance              | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | dut             | Top(fast)        | Module       | DU Instance              | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | u_asrt          | tb_assertion...  | Module       | DU Instance              | +acc=<n... +cover=<none> | 100.00%        |              | 9                | 9              | 0                 | 100.00%     |  |
| +                | u_cov           | tb_cov(fast)     | Module       | DU Instance              | +acc=<n... +cover=<none> | 100.00%        | 100.00%      |                  |                |                   |             |                                                                                     |
| #ALWAYS#13       | tb_top(fast)    | Process          | -            | +acc=<n...               |                          |                |              |                  |                |                   |             |                                                                                     |
| #INITIAL#44/#... | tb_top(fast)    | Process          | -            | +acc=<n...               |                          |                |              |                  |                |                   |             |                                                                                     |
| +                | tb_types_pkg    | tb_types_pk...   | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | aes_bfm_pkg     | aes_bfm_pk...    | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | aes_ref_pkg     | aes_ref_pk...    | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | stimuli_pkg     | stimuli_pkg(...  | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | scoreboard_pkg  | scoreboard_...   | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | tb_top_sv_unit  | tb_top_sv_u...   | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | std             | std              | VIPackage    | Package                  | +acc=<n... +cover=<none> |                |              |                  |                |                   |             |                                                                                     |
| +                | #vsim_capacity# |                  | Capacity     | Statistics               | +acc=<n...               |                |              |                  |                |                   |             |                                                                                     |

Figure 8: Verification Dashboard. The summary view confirms 100% Functional Coverage and 100% Assertion Pass Rate, indicating that all verification goals have been met.

### 4.3.1 Detailed Functional Coverage

The functional coverage model was designed to verify that the randomized stimuli effectively explored the entire operational space of the DUT. As shown in the coverage report, the environment achieved 100% Functional Coverage across all defined bins:

- **FSM State Traversal (cp\_state):** Confirmed that the design visited all architectural states (RESET, IDLE, SB, MC) and executed the full transition sequence without illegal jumps.
- **Input Data Distribution (cp\_input\_bucket):** Divided the 128-bit input space into 16 logical buckets. The report confirms that the randomizer generated values falling into every bucket, ensuring data diversity and preventing "local optimization" where only specific bit patterns are tested.
- **Protocol Corner Cases:**
  - **Reset-While-Busy (cp\_reset\_while\_busy):** Successfully captured the critical scenario where an asynchronous reset occurs during active processing. This verifies the FSM's ability to recover gracefully from an interrupted transaction.
  - **Start-in-Idle / SB-Valid-in-MC:** Verified control signal sampling windows, ensuring signals are asserted only in their valid architectural states.

```
# =====
# === Instance: /tb_top/u_cov
# === Design Unit: work.tb_cov
# =====
#
# Covergroup Coverage:
#   Covergroups          1      na      na    100.00%
#   Coverpoints/Crosses  6      na      na      na
#   Covergroup Bins      25     25      0    100.00%
#
# -----
```

| Covergroup                           | Metric  | Goal | Bins | Status  |
|--------------------------------------|---------|------|------|---------|
| TYPE /tb_top/u_cov/cg_main           | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 25      | 25   | -    |         |
| missing/total bins:                  | 0       | 25   | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_state                  | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 5       | 5    | -    |         |
| missing/total bins:                  | 0       | 5    | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_start_in_idle          | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 1       | 1    | -    |         |
| missing/total bins:                  | 0       | 1    | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_sbvalid_in_mc          | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 1       | 1    | -    |         |
| missing/total bins:                  | 0       | 1    | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_done_in_idle           | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 1       | 1    | -    |         |
| missing/total bins:                  | 0       | 1    | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_reset_while_busy       | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 1       | 1    | -    |         |
| missing/total bins:                  | 0       | 1    | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_input_bucket           | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 16      | 16   | -    |         |
| missing/total bins:                  | 0       | 16   | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Covergroup instance \tb_top/u_cov/cg | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 25      | 25   | -    |         |
| missing/total bins:                  | 0       | 25   | -    |         |
| % Hit:                               | 100.00% | 100  | -    |         |
| Coverpoint cp_state                  | 100.00% | 100  | -    | Covered |
| covered/total bins:                  | 5       | 5    | -    |         |

```
#
#
```

|   |                                      |         |     |   |         |
|---|--------------------------------------|---------|-----|---|---------|
| # | missing/total bins:                  | 0       | 1   | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | Coverpoint cp_input_bucket           | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 16      | 16  | - |         |
| # | missing/total bins:                  | 0       | 16  | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | Covergroup instance \tb_top/u_cov/cg | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 25      | 25  | - |         |
| # | missing/total bins:                  | 0       | 25  | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | Coverpoint cp_state                  | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 5       | 5   | - |         |
| # | missing/total bins:                  | 0       | 5   | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | bin reset                            | 6       | 1   | - | Covered |
| # | bin idle                             | 2147    | 1   | - | Covered |
| # | bin sb                               | 560     | 1   | - | Covered |
| # | bin mc                               | 559     | 1   | - | Covered |
| # | bin flow_idle_sb_mc_idle             | 559     | 1   | - | Covered |
| # | Coverpoint cp_start_in_idle          | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 1       | 1   | - |         |
| # | missing/total bins:                  | 0       | 1   | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | bin seen                             | 560     | 1   | - | Covered |
| # | Coverpoint cp_sbvalid_in_mc          | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 1       | 1   | - |         |
| # | missing/total bins:                  | 0       | 1   | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | bin seen                             | 559     | 1   | - | Covered |
| # | Coverpoint cp_done_in_idle           | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 1       | 1   | - |         |
| # | missing/total bins:                  | 0       | 1   | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | bin seen                             | 559     | 1   | - | Covered |
| # | Coverpoint cp_reset_while_busy       | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 1       | 1   | - |         |
| # | missing/total bins:                  | 0       | 1   | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | bin seen                             | 1       | 1   | - | Covered |
| # | Coverpoint cp_input_bucket           | 100.00% | 100 | - | Covered |
| # | covered/total bins:                  | 16      | 16  | - |         |
| # | missing/total bins:                  | 0       | 16  | - |         |
| # | % Hit:                               | 100.00% | 100 | - |         |
| # | bin bucket[0]                        | 29      | 1   | - | Covered |
| # | bin bucket[1]                        | 32      | 1   | - | Covered |
| # | bin bucket[2]                        | 29      | 1   | - | Covered |
| # | bin bucket[3]                        | 34      | 1   | - | Covered |
| # | bin bucket[4]                        | 41      | 1   | - | Covered |
| # | bin bucket[5]                        | 42      | 1   | - | Covered |
| # | bin bucket[6]                        | 31      | 1   | - | Covered |
| # | bin bucket[7]                        | 34      | 1   | - | Covered |
| # | bin bucket[8]                        | 35      | 1   | - | Covered |
| # | bin bucket[9]                        | 40      | 1   | - | Covered |
| # | bin bucket[10]                       | 35      | 1   | - | Covered |
| # | bin bucket[11]                       | 40      | 1   | - | Covered |
| # | bin bucket[12]                       | 43      | 1   | - | Covered |
| # | bin bucket[13]                       | 29      | 1   | - | Covered |
| # | bin bucket[14]                       | 30      | 1   | - | Covered |
| # | bin bucket[15]                       | 36      | 1   | - | Covered |
| # |                                      |         |     |   |         |

Figure 9: Detailed Functional Coverage Report. A breakdown of the specific covergroups, confirming that all FSM transitions, data buckets, and corner cases were exercised.



### 4.3.2 Protocol Validation (Assertions)

SystemVerilog Assertions (SVA) were active throughout the simulation to enforce strict protocol adherence. Unlike the Scoreboard (which checks data), these assertions act as active monitors for signal timing and legal sequencing. As seen in the assertion results, all 9 checkers passed successfully:

- **Handshake Sequencing:** Verified the causal relationship between control signals. For example, busy must assert exactly one cycle after start, and done must never assert while busy is low (except for the completion cycle).
- **Signal Integrity (Pulse Width):** Confirmed that sb\_valid and done flags operate as strictly single-cycle pulses. This is critical for downstream logic that relies on edge detection or pulse counting.
- **X-Propagation Safety:** Crucial for synthesis, assertions monitored the data buses (sb\_out, data\_out) to ensure no Unknown (X) or High-Impedance (Z) values propagated to the output ports whenever the validity flags were high. This guarantees clean, deterministic logic levels at the interface.

```
#
# ASSERTION RESULTS:
# -----
# Name                               File(Line)                               Failure   Pass
#                               Count                               Count
# -----
# /tb_top/u_asrt/assert__8
#                               TB/tb_assertions.sv(55)           0         1
# /tb_top/u_asrt/assert__7
#                               TB/tb_assertions.sv(50)           0         1
# /tb_top/u_asrt/assert__6
#                               TB/tb_assertions.sv(44)           0         1
# /tb_top/u_asrt/assert__5
#                               TB/tb_assertions.sv(39)           0         1
# /tb_top/u_asrt/assert__4
#                               TB/tb_assertions.sv(33)           0         1
# /tb_top/u_asrt/assert__3
#                               TB/tb_assertions.sv(27)           0         1
# /tb_top/u_asrt/assert__1
#                               TB/tb_assertions.sv(15)           0         1
# /tb_top/u_asrt/assert__0
#                               TB/tb_assertions.sv(10)           0         1
# /tb_top/u_asrt/assert__2
#                               TB/tb_assertions.sv(21)           0         1
#
# Total Coverage By Instance (filtered view): 100.00%
#
# End time: 00:05:50 on Dec 23,2025, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 10>
VSIM 10>
```

Figure 10: Assertion Results Table. Listings of all active protocol checks. The "Pass Count" confirms that every rule was triggered and validated successfully without violations.

### 4.3.3 Code Coverage Analysis

To validate the thoroughness of the verification environment, a structural Code Coverage analysis was performed. This metric ensures that the generated stimuli fully exercised the internal RTL structure, beyond just the functional protocol.

#### 4.3.3.1 Directed Coverage Strategy

While constrained-random verification is effective for control logic, relying solely on randomization is statistically inefficient for covering large look-up tables like the AES S-Box. To achieve 100% Statement Coverage of the substitution logic, a Deterministic Sweep Strategy was implemented. As illustrated in the code snippet below (Figure 11), the testbench executes a directed loop iterating through all 256 possible byte values (0x00–0xFF), where the loop variable *i* is replicated across all 16 byte-lanes of the 128-bit input vector ({16{b}}). This mechanism forces all 16 S-Box instances to process the same value simultaneously, effectively exercising every possible case item in the RTL within a minimal number of transactions.

```
// Code coverage sweep: exercise sbbox case items (0x00..0xFF)
// Drives all 16 bytes to the same value, so each sweep hits 16 sboxes.
for (int unsigned i = 0; i < 256; i++) begin
    logic [7:0] b;
    b = i[7:0];

    t.in_vec = {16{b}}; // 128-bit: {b,b,...} x16
    r = aes_ref_compute(t.in_vec);
    t.exp_sb = r.sb;
    t.exp_out = r.out;
    exp_mb.put(t);

    bfm.send_tx(t.in_vec, 0);
end
```

Figure 11: Directed Sweep Code. This loop forces all 16 S-Boxes to traverse every possible input value.

#### 4.3.3.2 Analysis of Results

As demonstrated in the simulation report, this hybrid strategy (Random + Directed Sweep) achieved exceptional coverage metrics:

- **Statement Coverage: 100%** – Confirms that every single line of the RTL code was executed (including all S-Box entries).
- **Toggle Coverage: 100%** – Verifies that every bit of every signal toggled.
- **Expression Coverage: 100%** – Confirms that all logical conditions were evaluated.
- **Branch Coverage: 99.61%** – Indicates that all control-flow paths were taken, with one justified exception.

```

VSIM 31> exec vcover report -codeAll -details -instance=/tb_top/dut -recursive /users_home/adishl/tb.ucdb
# Questasim-64 vcover 2022.3.1 Coverage Utility 2022.08 Aug 12 2022
# Start time: 12:50:09 on Jan 01,2026
# vcover report -codeAll -details -instance=/tb_top/dut -recursive /users_home/adishl/tb.ucdb
# ** Note: (vcover-19063) Please note that '-recursive' option's functionality is changed. For more information, please refer
# to the reference manual.
# ** Warning: (vcover-17229) '-details'/'-verbose' option is not supported with -recursive, ignoring it.
# Recursive Coverage Report Summary Data by instance
#
# =====
# === Instance: /tb_top/dut
# === Design Unit: work.Top
# =====
#   Enabled Coverage      Bins      Hits      Misses  Coverage
#   -----
#   Branches              4136      4120          16    99.61%
#   Expressions              6         6           0   100.00%
#   Statements            4157      4157           0   100.00%
#   Toggles                3642      3642           0   100.00%
#
#
# Total Coverage By Instance (filtered view): 99.90%
#
# End time: 12:50:09 on Jan 01,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 1

VSIM 32> exec vcover report -code b -details -instance=/tb_top/dut/u_sub_bytes/sub_Bytes[0]/s /users_home/adishl/tb.ucdb
# Questasim-64 vcover 2022.3.1 Coverage Utility 2022.08 Aug 12 2022
# Start time: 12:51:33 on Jan 01,2026
# vcover report -code b -details -instance=/tb_top/dut/u_sub_bytes/sub_Bytes[0]/s /users_home/adishl/tb.ucdb
# Coverage Report by instance with details
#
# =====
# === Instance: /tb_top/dut/u_sub_bytes/sub_Bytes[0]/s
# === Design Unit: work.sbox
# =====
# Branch Coverage:
#   Enabled Coverage      Bins      Hits      Misses  Coverage
#   -----
#   Branches              257      256           1    99.61%
#
# =====Branch Details=====
#
# Branch Coverage for instance /tb_top/dut/u_sub_bytes/sub_Bytes[0]/s
#
#   Line      Item      Count      Source
#   ---
#   Project : DDLS   Now: 32,720 ns   Delta: 0   12/20/2025 11:03:06 pm

```

Figure 12: Code Coverage Report. The summary confirms 100% coverage in Statements, Expressions, and Toggles. The granular analysis (bottom) confirms that the only missing branches are the unreachable 'default' cases in the S-Box instances.

### Root Cause Analysis for Missed Branches:

The report identifies exactly 16 missed branches. This number corresponds to the default case in the 16 instances of the sbox module.

- **The Calculation:** 1 Unreachable default branch × 16 S-Box instances = 16 Missed Branches.
- **Justification:** Since the S-Box input is strictly 8-bit (covering all 0-255 values via the sweep test), the default case is logically unreachable and exists only for synthesis safety.

**Conclusion:** The testbench achieved a total coverage metric of 99.90% Since the remaining gap corresponds solely to the unreachable S-Box defaults, this result effectively validates 100% of the functional logic within the design.

## 5. Synthesis

### 5.1 Synthesis Planning

Two synthesis runs were performed in Synopsys Design Compiler under the same timing objective of a 2.4 ns clock period (416 MHz target) to enable a controlled comparison between a timing-focused baseline and a low-power implementation. Both runs used the same RTL sources and the same technology environment, targeting the provided 90nm standard-cell library at the slow PVT corner (slow.db) and linking against DesignWare; therefore, any differences in the results are attributable to the selected optimization mode rather than changes in inputs or constraints.

The overall approach was timing-driven: the synthesis goal was defined by the clock requirement, and the tool was allowed to restructure the design to meet that target. The flow was intentionally planned as an A/B methodology:

- **Baseline (NOGATE):** Establishes a reference implementation optimized primarily for timing (and therefore serves as the baseline for area and power comparison).
- **Low-Power (GATE):** Applies clock-gating as an additional optimization step on top of timing optimization, in order to reduce switching activity while preserving the same timing target.

### Constraints

Synthesis was guided by a strict clock constraint and additional robustness settings intended to make the generated netlist suitable for downstream timing and gate-level analysis:

- **Clock target** of 2.4 ns, which defines the primary performance requirement for the entire design.
- **Clock uncertainty** of 0.10 ns, making timing closure more conservative and closer to realistic clock variation.
- **Hold-fixing enabled**, to improve robustness and reduce the risk of hold violations-particularly important when the design is later exported with delay annotation (SDF-based flows).
- **Netlist robustness fixes**, applied to avoid problematic multi-driven/constant net structures and produce a cleaner, more simulation- and implementation-friendly netlist.

### Optimization Strategy

To maximize the chances of meeting the aggressive timing target at the selected corner, synthesis relied on global and timing-centric optimizations:

- **Hierarchy flattening** was used to remove hierarchical boundaries before compilation. Conceptually, this allows the synthesizer to optimize across module interfaces (rather than being limited to local

per-block optimization), which can be important when the longest paths traverse multiple RTL blocks.

- **Register retiming** was enabled in both runs. Retiming allows the tool to rebalance sequential boundaries by moving registers across combinational logic where legal, with the goal of reducing the worst-case stage delay and improving the achievable clock frequency.
- **Clock gating (GATE run only)** was enabled to reduce dynamic power. The idea is to prevent unnecessary clock toggling in registers when their state is not required to change, by inserting integrated clock gating logic where valid enable conditions exist (using the defined gating style). This targets switching activity reduction while keeping the same timing closure objective.

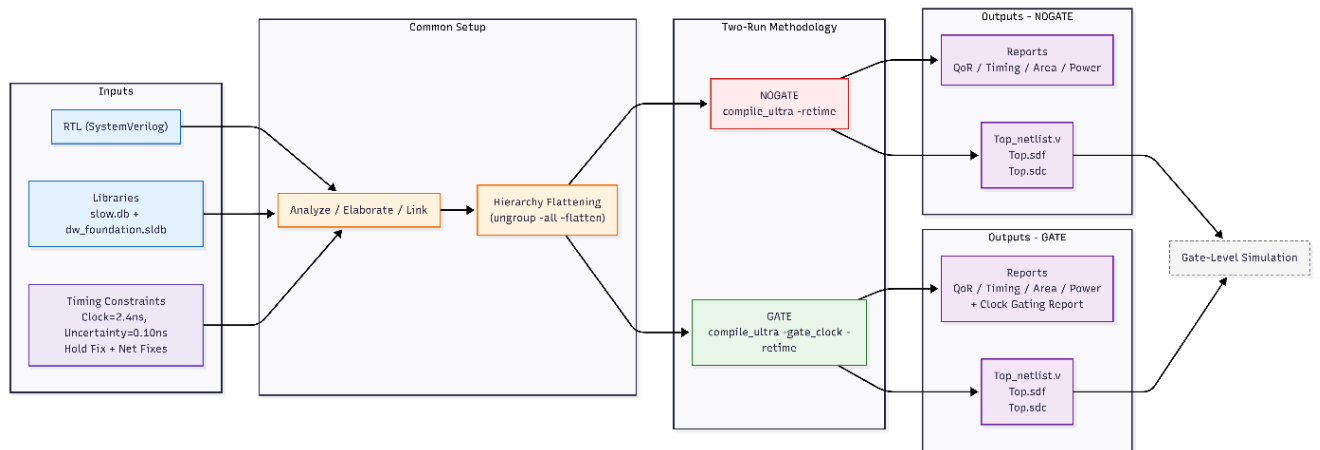


Figure 13: Synthesis flow overview

## 5.2 Timing Performance

Timing closure was evaluated at a target clock period of 2.4 ns, corresponding to a nominal operating frequency of 416 MHz. The design was synthesized in two configurations NOGATE (retiming only) and GATE (retiming with integrated clock gating) while maintaining the same timing objective and constraint set. Both configurations achieve full timing closure at the target constraint, with WNS = 0.00 ns (MET) and TNS = 0.00 ns, indicating no setup violations at the 2.4 ns requirement.

The QoR summary reflects a “boundary” closure (zero slack), therefore the result was also validated at the path level. The detailed timing reports confirm that the worst-case MAX (setup) paths terminate with slack (MET) = 0.00, showing that the frequency target is met on the true critical path and not only as an aggregate summary. This is important because it demonstrates that the design’s register-to-register timing budget is fully satisfied at the selected target period, with no residual negative slack across the design.

In the GATE configuration, the critical timing path explicitly includes clock-gating-related structures (clock-gate cell and latch-based gating elements), yet the reported slack remains 0.00 ns (MET). This provides direct evidence that clock gating was introduced without degrading timing closure, i.e., the low-power transformation did not introduce additional delay that would violate the 2.4 ns setup budget. Together with the NOGATE result, this confirms that both the baseline and the power-optimized implementations sustain the same 416 MHz timing target.

All timing results in this project were generated and evaluated using the slow library corner (slow.db). This corner is typically the most stringent condition for setup (max-delay) timing, since cell delays are maximized relative to typical/fast behavior. Therefore, achieving timing closure at 2.4 ns in the slow corner provides a conservative and robust validation of the 416 MHz target frequency. Hold timing was also checked using min-delay analysis within the same library context, serving as an additional robustness check alongside the primary setup closure at 2.4 ns.

### Timing Summary at 2.4 ns Target

| Flow          | Target Period | Frequency | Setup WNS / Slack | Setup TNS |
|---------------|---------------|-----------|-------------------|-----------|
| <b>NOGATE</b> | 2.4 ns        | 416 MHz   | 0.00 ns (MET)     | 0.00      |
| <b>GATE</b>   | 2.4 ns        | 416 MHz   | 0.00 ns (MET)     | 0.00      |

| Point                                                 | Incr  | Path   |
|-------------------------------------------------------|-------|--------|
| -----                                                 |       |        |
| clock clk (rise edge)                                 | 0.00  | 0.00   |
| clock network delay (ideal)                           | 0.00  | 0.00   |
| R_682/CK (DFFSX4)                                     | 0.00  | 0.00 r |
| R_682/Q (DFFSX4)                                      | 0.44  | 0.44 f |
| U7509/Y (NOR2X2)                                      | 0.17  | 0.61 r |
| U10092/Y (NAND2X4)                                    | 0.08  | 0.69 f |
| U10093/Y (INVX4)                                      | 0.10  | 0.79 r |
| U10123/Y (NAND2X1)                                    | 0.12  | 0.91 f |
| U8965/Y (NAND2X1)                                     | 0.16  | 1.07 r |
| U12439/Y (NOR2X1)                                     | 0.08  | 1.15 f |
| U8964/Y (NAND2BX1)                                    | 0.13  | 1.28 r |
| U12444/Y (NOR3X2)                                     | 0.07  | 1.35 f |
| U7554/Y (CLKINVX2)                                    | 0.09  | 1.44 r |
| U12451/Y (NOR4X2)                                     | 0.06  | 1.50 f |
| U12465/Y (AOI211X1)                                   | 0.28  | 1.78 r |
| U12485/Y (NOR3X1)                                     | 0.08  | 1.86 f |
| U12486/Y (OAI2BB1XL)                                  | 0.18  | 2.03 f |
| u_sub_bytes/u_output_reg/data_out_reg[72]/D (DFFSX1)  |       |        |
| data arrival time                                     | 0.00  | 2.03 f |
| data arrival time                                     |       | 2.03   |
| -----                                                 |       |        |
| clock clk (rise edge)                                 | 2.40  | 2.40   |
| clock network delay (ideal)                           | 0.00  | 2.40   |
| clock uncertainty                                     | -0.10 | 2.30   |
| u_sub_bytes/u_output_reg/data_out_reg[72]/CK (DFFSX1) |       |        |
| library setup time                                    | 0.00  | 2.30 r |
| data required time                                    | -0.26 | 2.04   |
| data required time                                    |       | 2.04   |
| -----                                                 |       |        |
| data required time                                    |       | 2.04   |
| data arrival time                                     |       | -2.03  |
| -----                                                 |       |        |
| slack (MET)                                           |       | 0.00   |

Figure 14: Worst-case setup timing path for the NOGATE implementation at 2.4 ns target period, showing slack (MET) = 0.00 ns (timing closure achieved at 416 MHz).

| Point                                                                            | Incr  | Path   |
|----------------------------------------------------------------------------------|-------|--------|
| -----                                                                            |       |        |
| clock clk (rise edge)                                                            | 0.00  | 0.00   |
| clock network delay (ideal)                                                      | 0.00  | 0.00   |
| u_control_fsm/u_state_reg/data_out_reg[1]/CK (DFFSX1)                            |       |        |
| u_control_fsm/u_state_reg/data_out_reg[1]/QN (DFFSX1)                            | 0.00  | 0.00 r |
| U9881/Y (BUFX12)                                                                 | 0.54  | 0.54 f |
| U14710/Y (NAND2X1)                                                               | 0.29  | 0.83 f |
| U14711/Y (INVX1)                                                                 | 0.14  | 0.97 r |
| clk_gate_u_mix_columns/u_output_reg/data_out_reg/EN (SNPS_CLOCK_GATE_HIGH_Top_1) | 0.05  | 1.02 f |
| clk_gate_u_mix_columns/u_output_reg/data_out_reg/test_or/Y (OR2X2)               | 0.00  | 1.02 f |
| clk_gate_u_mix_columns/u_output_reg/data_out_reg/test_or/Y (OR2X2)               | 0.19  | 1.21 f |
| clk_gate_u_mix_columns/u_output_reg/data_out_reg/latch/D (TLATNX1)               |       |        |
| data arrival time                                                                | 0.00  | 1.21 f |
| data arrival time                                                                |       | 1.21   |
| -----                                                                            |       |        |
| clock clk (fall edge)                                                            | 1.20  | 1.20   |
| clock network delay (ideal)                                                      | 0.00  | 1.20   |
| clock uncertainty                                                                | -0.10 | 1.10   |
| clk_gate_u_mix_columns/u_output_reg/data_out_reg/latch/GN (TLATNX1)              |       |        |
| time borrowed from endpoint                                                      | 0.00  | 1.10 f |
| data required time                                                               | 0.11  | 1.21   |
| data required time                                                               |       | 1.21   |
| -----                                                                            |       |        |
| data required time                                                               |       | 1.21   |
| data arrival time                                                                |       | -1.21  |
| -----                                                                            |       |        |
| slack (MET)                                                                      |       | 0.00   |

Figure 15: Worst-case setup timing path for the GATE implementation at 2.4 ns target period. The path includes clock-gating-related elements (e.g., clock-gate/latch structure), while still meeting timing with slack (MET) = 0.00 ns, demonstrating no timing degradation due to clock gating.

### 5.3 Power & Area Efficiency (Clock Gating Analysis)

Power and area were measured for both synthesis configurations (NOGATE vs. GATE) under the same timing target (2.4 ns). Power numbers were taken from Design Compiler report\_power using the slow.db library at VDD = 1.62 V, and area results were taken from report\_area. Area is reported using total cell area, which is a consistent metric across both runs.

#### Gate vs. NoGate Summary

Change indicates the percentage difference of the GATE run relative to the NOGATE baseline

| Metric                     | NOGATE       | GATE             | Change |
|----------------------------|--------------|------------------|--------|
| Total Dynamic Power [mW]   | 21.9355      | 4.5192           | -79.4% |
| Clock Network Power [mW]   | 17.0885      | 1.3870           | -91.9% |
| Cell Leakage Power [uW]    | 5.2286       | 3.9821           | -23.8% |
| Total Cell Area            | 172,769.8907 | 145,107.5485     | -16.0% |
| Cells                      | 9,430        | 8,479            | -10.1% |
| Sequential Cells           | 783          | 589              | -24.8% |
| Gated Registers (GATE run) | -            | 99.49% (582/585) | -      |

#### Discussion

In the baseline NOGATE run, most of the dynamic power is consumed by the clock distribution activity: clock\_network contributes 17.0885 mW out of 21.9355 mW total dynamic power. After enabling clock gating (GATE run), the clock network power drops sharply to 1.3870 mW, which drives the overall dynamic power down to 4.5192 mW. This shows that clock gating achieves a major reduction in switching power without changing the timing target.

The clock-gating report also shows very high coverage: 582 out of 585 registers are gated (99.49%), with only 3 registers remaining ungated, indicating that clock gating was broadly applied across the design.

In terms of implementation cost, the GATE run does not increase area in this synthesis outcome. The results show a lower total cell area and fewer mapped cells compared to the NOGATE baseline.



| Power Group   | Internal Power | Switching Power | Leakage Power | Total Power | ( % )     | Attrs |
|---------------|----------------|-----------------|---------------|-------------|-----------|-------|
| io_pad        | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| memory        | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| black_box     | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| clock_network | 1.2294         | 0.1576          | 7.0340e+03    | 1.3870      | ( 30.66%) | i     |
| register      | 1.2949         | 0.1828          | 1.2333e+06    | 1.4790      | ( 32.70%) |       |
| sequential    | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| combinational | 0.8217         | 0.8328          | 2.7418e+06    | 1.6572      | ( 36.64%) |       |
| Total         | 3.3460 mW      | 1.1732 mW       | 3.9821e+06 pW | 4.5232 mW   |           |       |
| 1             |                |                 |               |             |           |       |

Figure 16: NOGATE Power Breakdown

| Power Group   | Internal Power | Switching Power | Leakage Power | Total Power | ( % )     | Attrs |
|---------------|----------------|-----------------|---------------|-------------|-----------|-------|
| io_pad        | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| memory        | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| black_box     | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| clock_network | 17.0885        | 0.0000          | 0.0000        | 17.0885     | ( 77.88%) | i     |
| register      | 1.9842         | 0.3158          | 1.7546e+06    | 2.3017      | ( 10.49%) |       |
| sequential    | 0.0000         | 0.0000          | 0.0000        | 0.0000      | ( 0.00%)  |       |
| combinational | 1.2259         | 1.3211          | 3.4740e+06    | 2.5505      | ( 11.62%) |       |
| Total         | 20.2986 mW     | 1.6369 mW       | 5.2286e+06 pW | 21.9407 mW  |           |       |
| 1             |                |                 |               |             |           |       |

Figure 17: GATE Power Breakdown

| Hierarchical area distribution |                  |               |                 |                   |             |        |
|--------------------------------|------------------|---------------|-----------------|-------------------|-------------|--------|
| Hierarchical cell              | Global cell area |               | Local cell area |                   |             | Design |
|                                | Absolute Total   | Percent Total | Combi-national  | Noncombi-national | Black-boxes |        |
| Top                            | 172769.8907      | 100.0         | 116903.0032     | 55866.8874        | 0.0000      | Top    |
| Total                          |                  |               | 116903.0032     | 55866.8874        | 0.0000      |        |
| 1                              |                  |               |                 |                   |             |        |

Figure 18: Area summary NOGATE

| Hierarchical area distribution                   |                  |               |                 |                   |             |                            |
|--------------------------------------------------|------------------|---------------|-----------------|-------------------|-------------|----------------------------|
| Hierarchical cell                                | Global cell area |               | Local cell area |                   |             | Design                     |
|                                                  | Absolute Total   | Percent Total | Combi-national  | Noncombi-national | Black-boxes |                            |
| Top                                              | 145107.5485      | 100.0         | 103886.8000     | 40967.9420        | 0.0000      | Top                        |
| clk_gate_u_control_fsm/u_state_reg/data_out_reg  | 63.2016          | 0.0           | 26.6112         | 36.5904           | 0.0000      | SNPS_CLOCK_GATE_HIGH_Top_0 |
| clk_gate_u_input_reg/data_out_reg                | 63.2016          | 0.0           | 26.6112         | 36.5904           | 0.0000      | SNPS_CLOCK_GATE_HIGH_Top_3 |
| clk_gate_u_mix_columns/u_output_reg/data_out_reg | 63.2016          | 0.0           | 26.6112         | 36.5904           | 0.0000      | SNPS_CLOCK_GATE_HIGH_Top_1 |
| clk_gate_u_sub_bytes/u_output_reg/data_out_reg   | 63.2016          | 0.0           | 26.6112         | 36.5904           | 0.0000      | SNPS_CLOCK_GATE_HIGH_Top_2 |
| Total                                            |                  |               | 103993.2448     | 41114.3036        | 0.0000      |                            |

Figure 19: Area summary GATE

| Clock Gating Report by Origin                 |                     |
|-----------------------------------------------|---------------------|
|                                               | Actual (%)<br>Count |
| Number of tool-inserted clock gating elements | 4 (100.00%)         |
| Number of pre-existing clock gating elements  | 0 (0.00%)           |
| Number of gated registers                     | 582 (99.49%)        |
| Number of tool-inserted gated registers       | 582 (99.49%)        |
| Number of pre-existing gated registers        | 0 (0.00%)           |
| Number of ungated registers                   | 3 (0.51%)           |
| Number of registers                           | 585                 |

Figure 20: Clock Gating Report

## 5.4 Gate-Level Simulation (GLS)

To validate the post-synthesis implementation, we performed gate-level simulation (GLS) with SDF back-annotation, in order to confirm that the synthesized design remains functionally correct under realistic standard-cell and interconnect delays. SDF was annotated in both worst-case (max delays) and best-case (min delays) modes, enabling validation under conservative setup timing as well as hold-sensitive conditions.

We ran GLS on two post-synthesis netlists:

- NOGATE: synthesized netlist without clock-gating insertion
- GATE: synthesized netlist with clock-gating insertion

Both variants were simulated using the same SystemVerilog verification environment (identical testbench, stimuli, reference model, and scoreboard), enabling a direct functional comparison under identical conditions.

### Results

In both GLS runs:

- SDF annotation was applied successfully
- The simulation completed with no errors
- The verification environment reported a clear PASS indication

### Conclusion

The GLS results confirm that:

- The synthesized netlist preserves the intended AES functional behavior.
- The design operates correctly under back-annotated gate-level timing.
- Clock-gating insertion does not affect functionality, as both netlists are functionally equivalent under the same verification flow.

```

# Loading instances from /users_home/adishl/DDLS PROJECT FINAL/gate_level/nogate/Top.sdf
# Loading timing data from /users_home/adishl/DDLS PROJECT FINAL/gate_level/nogate/Top.sdf
# ** Note: (vsim-3587) SDF Backannotation Successfully Completed.
#   Time: 0 ps   Iteration: 0   Instance: /tb_top File: /users_home/adishl/DDLS PROJECT FINAL/gate_level_TB/tb_top.sv

VSIM 53> run -all
# TB COMPLETE | txns=559 pass=1118 fail=0
# ** Note: $stop : /users_home/adishl/DDLS PROJECT FINAL/gate_level_TB/tb_top.sv(65)
#   Time: 32720 ns   Iteration: 0   Instance: /tb_top
# Break in Module tb_top at /users_home/adishl/DDLS PROJECT FINAL/gate_level_TB/tb_top.sv line 65

```

Figure 21: NOGATE GLS Transcript confirmed SDF applied, all transactions PASS, and no errors

```

# Loading instances from /users_home/adishl/DDLS PROJECT FINAL/gate_level/gate/Top.sdf
# Loading timing data from /users_home/adishl/DDLS PROJECT FINAL/gate_level/gate/Top.sdf
# ** Note: (vsim-3587) SDF Backannotation Successfully Completed.
#   Time: 0 ps   Iteration: 0   Instance: /tb_top File: /users_home/adishl/DDLS PROJECT FINAL/gate_level_TB/tb_top.sv

VSIM 23> run -all
# TB COMPLETE | txns=559 pass=1118 fail=0
# ** Note: $stop : /users_home/adishl/DDLS PROJECT FINAL/gate_level_TB/tb_top.sv(65)
#   Time: 32720 ns   Iteration: 0   Instance: /tb_top
# Break in Module tb_top at /users_home/adishl/DDLS PROJECT FINAL/gate_level_TB/tb_top.sv line 65

VSIM 24>

```

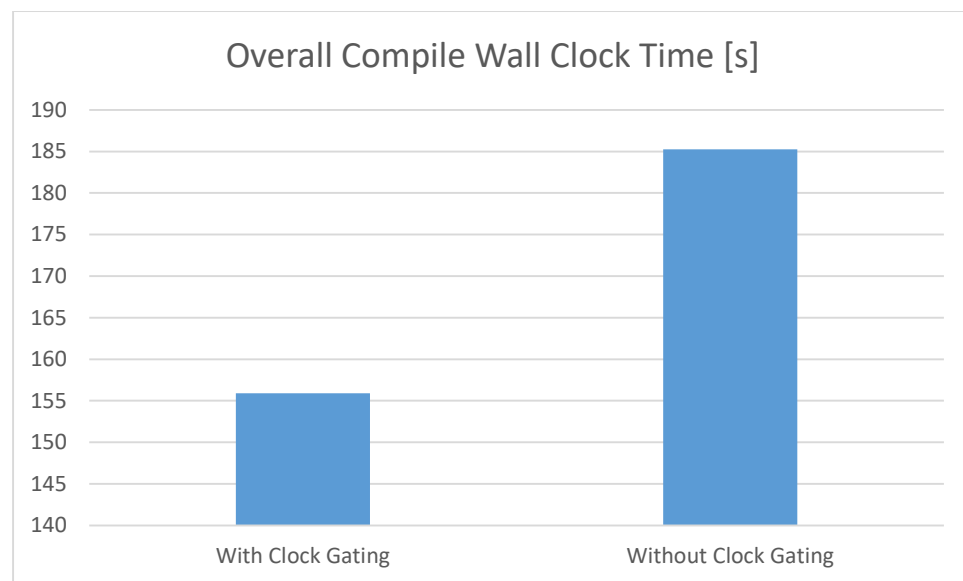
Figure 22: GATE GLS Transcript confirmed SDF applied, all transactions PASS, and no errors

## 5.5 Runtime and Resource Utilization

### Synthesis Runtime

The synthesis flow was executed twice under identical conditions, once with clock gating enabled and once without clock gating. Runtime metrics were extracted directly from the Design Compiler QoR reports using the *Overall Compile Time* and *Overall Compile Wall Clock Time* fields.

Synthesis Runtime Comparison:



The results show that enabling clock gating reduces the overall synthesis runtime. This behavior is consistent with the fact that clock gating simplifies switching activity and enables more efficient optimization during synthesis.

```
Compile CPU Statistics
-----
Resource Sharing:          52.15
Logic Optimization:       24.09
Mapping Optimization:     13.79
-----
Overall Compile Time:      155.42
Overall Compile Wall Clock Time: 155.88
```

Figure 23: Compile CPU Statistics (GATE run)

```

Compile CPU Statistics
-----
Resource Sharing:           56.97
Logic Optimization:        32.77
Mapping Optimization:       27.09
-----
Overall Compile Time:       184.80
Overall Compile Wall Clock Time: 185.25

```

Figure 24: Compile CPU Statistics (NOGATE run)

Memory utilization was monitored at the system level throughout the synthesis execution, and all runs completed successfully without memory pressure, swapping activity, or any indication of resource starvation. Available RAM remained high during the flow, and the tool did not exhibit behavior that typically correlates with memory bottlenecks (e.g., heavy paging or degraded responsiveness), indicating that memory usage was not a limiting factor in this experiment. Regarding `get_clocks`, it is used only in the constraints stage for clock definition and uncertainty setup; this command is evaluated once during constraint elaboration and does not scale with design size, iteration count, or optimization effort, so its contribution to overall runtime and peak memory is effectively negligible. Consequently, no measurable difference in runtime or memory consumption can be attributed specifically to the presence or absence of `get_clocks` in the flow. In summary, the results indicate that clock gating reduces overall synthesis runtime, memory resources remain stable and non-limiting across all runs, and `get_clocks` does not have a measurable impact on runtime or memory utilization in this setup.

## 6. Conclusion

This project successfully covered the full VLSI front-end flow of an AES-128 core transformation accelerator, spanning from architectural definition to gate-level implementation. The design phase delivered a robust RTL architecture featuring an FSM-controlled pipelined datapath (SubBytes/MixColumns) and a clean top-level handshake protocol.

**Verification Results:** The design was validated using a comprehensive SystemVerilog verification environment. By employing a layered testbench architecture (BFM, golden reference model, and scoreboard), complemented by assertions and functional/code coverage, the project achieved automated correctness checking across a 559-transaction regression. The results demonstrated 100% functional coverage and a 100% assertion pass rate, with code coverage confirming that all reachable logic was exercised (excluding only structurally unreachable S-Box defaults).

**Implementation & Physical Results:** A comparative synthesis study (NOGATE vs. GATE) was conducted under a 2.4 ns timing target. Both configurations achieved timing closure at 416 MHz in the slow process corner. The power analysis highlighted the impact of Integrated Clock Gating, reducing total dynamic power from 21.94 mW to 4.52 mW (79.4%) and clock network power by 91.9%, without compromising performance. *(Power values are reported under identical tool assumptions for both runs.)* Finally, Gate-Level Simulation (GLS) with SDF back-annotation (max/min) demonstrated functional consistency of the synthesized netlists with the RTL under the executed verification regressions. In addition, runtime measurements showed that the clock-gated (GATE) synthesis completed faster than the non-gated (NOGATE) flow, while memory usage remained stable and non-limiting throughout the runs.